ScamGuard: AI-Based Online Scam and Fraud Detection System



PROJECT REPORT

Submitted by

Bhavya Goyal (1BM23CS063)
Priyam Thakur (1BM23CS252)
Samir Chaudhary (1BM23CS294)
Sana Subodh (1BM23CS296)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Under the Guidance of
Dr. Kayarvizhy N
Professor, Department of CSE, BMSCE



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU - 560019
Academic Year - 2026–2027

Certificate

This is to certify that the project report entitled **"ScamGuard: AI-Based Online Scam and Fraud Detection System"** is a bonafide work carried out by **Bhavya Goyal (USN: 1BM23CS063)**, **Priyam Thakur (USN: 1BM23CS252)**, **Samir Chaudhary (USN: 1BM23CS294)** and **Sana Subodh (USN: 1BM23CS296)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in **Computer Science and Engineering**, under my supervision during the academic year 2026–2027.

Guide Signature	HoD signature	Principal Signature
(Dr. Kayarvizhy N)	Dr. Kavitha Sooda	Dr. Bheemsha Arya

Name and signature of Examiners along with date

Examiner 1

Examiner 2

Abstract

Fraudulent SMS messages are one of the cheapest ways to reach a phone user in India. They imitate bank KYC reminders, electricity disconnection notices, parcel delivery updates, traffic challans and job offers, and they increasingly arrive in Hindi or in a mix of Hindi and English. Public smishing collections contain only the messages that users chose to report, so they offer no verified genuine class, and a detector trained on them learns to distrust every bank-like message. ScamGuard was built to separate Indian scam messages from genuine ones with few false alarms and to explain each verdict.

We built ScamGuard-IN-2026, a corpus of 11,399 messages assembled from the India-relevant part of a 2025 research collection of user-reported smishing, a public set of real Indian SMS, scam texts quoted in 2025–26 government advisories and news reports, and synthetic messages that cover the newest scam types and are used for training only. Reported messages are not automatically scams, so we checked all 2,922 groups of near-identical reported messages, by rule or by reading them. About one in five retained reports turned out to be a genuine bank alert, OTP message, order update or advertisement. Messages were grouped into near-duplicate campaigns with MinHash, and the data was split so that no campaign appears in both training and testing.

The detector is a logistic-regression classifier over word and character TF-IDF features, combined with 31 interpretable intent patterns that explain each verdict by highlighting the exact phrases that make a message risky. On 1,874 held-out real messages from unseen campaigns, the classifier reaches an F1-score of 98.1% with a 2.6% false-alarm rate, and the deployed system reaches 97.1% accuracy (95% confidence interval 96.3–97.8%) with a 3.2% false-alarm rate. Training on unverified report labels instead of audited labels raises the false-alarm rate to 34.0%. The detector is part of a working web application with a FastAPI back end and a Next.js front end, which offers a quick check without sign-in, a full analysis with phrase-level explanations, an AI copilot, safety tools for QR codes, UPI requests and files, scan history and analytics. The application is deployed on Render, and every number in this report can be reproduced from the project repository.

Keywords: scam detection, smishing, SMS fraud, machine learning, natural language processing, label noise, data leakage, explainable AI

DECLARATION

We hereby declare that the Major Project Phase-2 work entitled "ScamGuard: AI-Based Online Scam and Fraud Detection System" is a bonafide work and has been carried out by us under the guidance of Dr. Kayarvizhy N, Professor, Department of Computer Science and Engineering, B.M.S. College of Engineering, Bengaluru, in partial fulfillment of the requirements of the degree of Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi. We further declare that, to the best of our knowledge and belief, this project has not been submitted either in part or in full to any other university for the award of any degree.

Candidate details:

SL. NO.	Student Name	USN	Student's Signature
1	Bhavya Goyal	1BM23CS063	
2	Priyam Thakur	1BM23CS252	
3	Samir Chaudhary	1BM23CS294	
4	Sana Subodh	1BM23CS296	

Place: Bengaluru
Date:

Certified that these candidates are students of Computer Science and Engineering Department of B.M.S. College of Engineering. They have carried out the project work titled "ScamGuard: AI-Based Online Scam and Fraud Detection System" as Major Project Phase-2 work. It is in partial fulfillment for completing the requirement for the award of B.E. degree by VTU. The work is original and we duly certify the same.

Guide Name: Dr. Kayarvizhy N
Date:
Signature

Acknowledgment

We are grateful to our guide, Dr. Kayarvizhy N, Professor, Department of Computer Science and Engineering, for steering this project towards a question that could be answered with evidence, and for insisting that the dataset reflect the messages people in India actually receive today. Her guidance on building a dataset that reflects current Indian scams shaped the audited corpus described here.

We thank Dr. Kavitha Sooda, Head of the Department of Computer Science and Engineering, and Dr. Bheemsha Arya, Principal, B.M.S. College of Engineering, for the facilities and the academic environment that made this work possible. We also thank the faculty members who reviewed our phase presentations and pointed out gaps in our evaluation.

We acknowledge the researchers who released the IMC 2025 smishing collection and the India Spam SMS collection under open licences; without them this study would have needed far more data collection. Finally, we thank our families and friends, several of whom forwarded the suspicious messages that shaped our test cases.

Contents

1 Introduction **1**
1.1 Overview 1
1.2 Motivation 1
1.3 Objective 2
1.4 Scope 2
1.5 SDG Justification 3
1.6 Existing System 3
1.7 Proposed System 4
1.8 Work Plan 4

2 Literature Survey **5**
2.1 Overview 5
2.2 Detailed Survey 5
2.3 Research Gap Identification 7
2.4 Mapping of Objectives with Research Gap Identification 7

3 Software and Hardware Requirement Specification **8**
3.1 Functional Requirements 8
3.2 Non-functional Requirements 9
3.3 Hardware Requirements 9
3.4 Software Requirements 10
3.5 Cost Estimation 10

4 Methodology **11**
4.1 Tools and Techniques 11
4.2 Data Set With Explanation 11
4.2.1 Need for an Indian dataset 11
4.2.2 Sources 12
4.2.3 Attributes 12
4.2.4 Label audit 13
4.2.5 Campaigns and splits 13
4.3 Flow Chart Of Methodology From Start To End 14

4.4 Explanation Of Methodology In Details With Equations And Formulae . 15
4.4.1 Problem definition 15
4.4.2 Preprocessing 15
4.4.3 Feature extraction 16
4.4.4 Classifier 16
4.4.5 Intent evidence and fusion 16
4.4.6 Evaluation metrics 17
4.4.7 Inference procedure 17

5 Results and Discussion 18
5.1 Experimental setup 18
5.2 Performance of the deployed system 18
5.3 Comparison of models and features 19
5.4 Effect of the label audit 20
5.5 Effect of the split and of synthetic data 20
5.6 Value of the intent rules 21
5.7 Operating point and errors 21
5.8 Application snapshots 22

6 Conclusion and Future Work 23
6.1 Conclusion 23
6.2 Future Work 23

REFERENCES 24
Appendix A: Snapshots 26
Appendix B: Details of list of publications related to this project 32
Appendix C: Details of Patent 33
Appendix D: Details of Funding 34
Appendix E: POs and PSOs Mapped 35
Appendix F: Similarity report, AI Generated report 37

List of Figures

4.1 Flow of the methodology 14
4.2 Architecture of the deployed system. Shaded parts run offline. 15

5.1 Accuracy, precision, recall and F1-score on the real test set (vertical axis starts at 95%) 19
5.2 False-alarm rate with report labels and with audited labels 20
5.3 ROC and precision-recall curves on the real test set 21
5.4 (a) Confusion matrix of the deployed system; (b) recall and false-alarm rate by threshold 22

A.1 Home page with the quick check 26
A.2 Quick check of a fake KYC message: verdict, matched tactics, known pattern and Scam X-Ray 26
A.3 Quick check of a genuine operator notice 27
A.4 Full analysis of a scam message for a signed-in user 27
A.5 Scam X-Ray: each risky phrase placed in the hook, pressure, request and cash-out stages 28
A.6 TrustLens: scam journey, recommended actions and how a genuine message differs 28
A.7 Dashboard with priority findings and recent scans 29
A.8 Scan history with search, filters and export 29
A.9 Analytics: verdict share, distribution and score trend 30
A.10 Safety tools: QR code check, before-you-pay, file check and chat timeline 30
A.11 Scam simulator for practice 31

List of Tables

1.1 Work plan 4

2.1 Summary of related work 6
2.2 Mapping of objectives to research gaps 7

3.1 Hardware requirements 9
3.2 Software requirements 10

4.1 Sources of ScamGuard-IN-2026 12
4.2 Dataset splits 13

5.1 Real test set, 1,874 messages from unseen campaigns (%; FAR = false-alarm rate) 18
5.2 Classifiers on the real test set (%); "Challenge" is F1 on the 259 team-written messages 19
5.3 Effect of the training data (logistic regression, word + char, %) 20
5.4 Classifier alone and with fusion (%) 21

E.1 Programme Outcomes Mapping with Project 35
E.2 Programme Specific Outcomes Mapping with Project 36

Chapter 1

Introduction

1.1 Overview

A scam SMS rarely looks like a scam. It looks like a message from a bank asking for a quick KYC update, from an electricity board warning of disconnection tonight, from a courier holding a parcel for a small fee, or from the traffic police with a challan to pay through an attached app. Fraudsters copy the format of genuine service messages because people have learned to trust those formats. A detector has to tell the two apart even though they share most of their vocabulary: both mention OTPs, accounts, KYC, links and deadlines.

ScamGuard is a web application that does this for a single message pasted by the user. It returns a verdict, a risk score and, more importantly, the reasons: the phrases that make the message risky, the stage of the scam each phrase belongs to, what a genuine message from the same organisation would do instead, and what the user should do next. Behind the application is a text classifier and a layer of interpretable rules, both trained and evaluated on a dataset that we built for Indian messages.

This report describes the problem, the dataset, the detection method, the application and the evaluation. The research part of the project is also written up as a separate paper.

1.2 Motivation

Three observations motivated the project.

First, the scale and the Indian focus of the problem. A 2025 measurement study of more than 64,000 public smishing reports found that most attributable sender numbers belonged to Indian mobile operators [1]. The common Indian scams are well documented in advisories, but users still have to judge each message on their own.

Second, the cost of false alarms. Genuine Indian bank alerts, OTP messages and operator notices use the same words as scams: *account*, *KYC*, *block*, *OTP*, *link*. Public smishing data contains only reported messages, and many of those reports are themselves genuine messages. A tool that cries wolf on real bank messages does damage, because

it teaches users to ignore warnings.

Third, the lack of explanation in most tools. A bare "spam" label gives the user nothing to act on. Users need to see which part of the message is the trap, so that they can recognise the next one without a tool.

1.3 Objective

The objectives of the project are:

1. To build a labelled dataset of Indian scam and genuine SMS that reflects current scam types and whose labels have been checked, rather than taken from user reports at face value.
2. To evaluate lightweight text classifiers on that dataset under a protocol that prevents near-duplicate messages from leaking between training and testing.
3. To combine the best classifier with interpretable intent rules that explain each verdict at phrase level, and to measure honestly whether the rules also improve accuracy.
4. To deliver the detector in a usable, deployed web application with supporting safety tools, history and analytics.
5. To make every reported result reproducible from the project repository.

1.4 Scope

The system analyses one message at a time, as pasted by the user, in English, Hindi (Devanagari script) or romanised Hindi. It classifies the message as *scam* or *legitimate*. A scam is a message that tries to obtain money, a secret such as an OTP or PIN, personal data, an app installation or a call to a private number by deception. Genuine advertisements are legitimate. The application also accepts URLs, e-mails, screenshots, PDFs and QR codes, which it converts to text or checks with separate tools.

The project does not intercept messages on the phone, does not use sender metadata or conversation history in the classifier, and does not cover Indian languages other than Hindi. The risk score is a ranking signal, not a calibrated probability, and the verdict is advice, not proof of fraud.

1.5 SDG Justification

The project addresses the following United Nations Sustainable Development Goals.

- **SDG 16 (Peace, Justice and Strong Institutions), target 16.4 on reducing illicit financial flows.** Scam messages are the entry point of many financial frauds. Helping users recognise them before they pay or share an OTP reduces the money that reaches fraudsters, and the application points users to the official reporting channels (helpline 1930 and cybercrime.gov.in).
- **SDG 8 (Decent Work and Economic Growth), target 8.10 on access to financial services.** Digital payments only widen financial access if people can trust them. A tool that separates genuine bank and UPI messages from imitations supports that trust, especially for first-time digital users.
- **SDG 9 (Industry, Innovation and Infrastructure), target 9.c on access to ICT.** The detector is small enough to run on free cloud infrastructure and answers in milliseconds, so it can be offered at no cost to users.
- **SDG 10 (Reduced Inequalities).** Support for Hindi messages and a simplified "elder mode" interface aim at users who are most often targeted and least served by English-only tools.

1.6 Existing System

Existing approaches fall into three groups.

- **Operator and regulator measures.** Telecom operators filter bulk traffic and register commercial senders, and users can report unwanted messages to the 1909 DND service or to the cyber-crime portal. These measures act on senders and volumes; they do not help a user judge a message that has already arrived from an ordinary mobile number.
- **Rule-based and keyword filters.** Phone apps and simple filters flag words such as "lottery" or "KYC". They are easy to explain but flag many genuine messages that use the same words and miss scams that avoid them.
- **Machine-learning filters in research.** Classifiers such as Naive Bayes, SVM, CNN and LSTM report high accuracy, but mostly on promotional spam, with random train-test splits and without a genuine Indian class [5, 6, 9]. Newer report-based corpora contain only reported messages [10, 1].

1.7 Proposed System

The proposed system has three parts.

1. **A dataset, ScamGuard-IN-2026.** Real Indian reported scams, real Indian genuine messages, 2025–26 advisory texts and synthetic messages for the newest scam types, with every reported-message group checked and the data split by scam campaign.
2. **A detector.** Logistic regression over word and character TF-IDF features, which handles Hindi and obfuscated spellings, combined with interpretable intent rules and genuine-message guards that explain each verdict.
3. **A complete application.** A deployed web application with a quick check, a full analysis with Scam X-Ray and TrustLens explanations, an AI copilot, safety tools, history, analytics and an automated daily check of the live site.

1.8 Work Plan

Table 1.1 shows the work plan across the two project phases.

Table 1.1: Work plan

Phase	Activity	Outcome
1	Problem study and literature survey	Research gap: no verified Indian genuine class, leakage-prone evaluation
1	Requirement analysis and design	System architecture, data model and user flows
1	Application architecture	FastAPI back end, Next.js front end, model registry, database
2	Explanation layer	Intent rules, Scam X-Ray, TrustLens, retrieval of known patterns
2	Dataset construction	ScamGuard-IN-2026 with label audit and campaign split
2	Model selection and experiments	Final model; leakage, label-audit, training-data and fusion experiments
2	Deployment and testing	Render deployment, 818 automated tests, daily live-site check
2	Documentation	Report, research paper, presentation

Chapter 2

Literature Survey

2.1 Overview

We reviewed work on SMS spam corpora, smishing detection, report-based measurement studies and general machine-learning methodology for label noise and data leakage. Each reference below was checked against its publisher page, DOI record or arXiv entry. Results quoted are those stated by the authors; where a value is not stated we say so.

2.2 Detailed Survey

SMS Spam Collection. Almeida, Gómez Hidalgo and Yamakami released the SMS Spam Collection in 2011 [4]: 5,574 English messages, 747 of them spam, mostly from the UK. In a follow-up they compared many classifiers and tokenisers and preferred a linear SVM, which reached 97.64% accuracy while blocking only 0.18% of legitimate messages [5]. The corpus is clean and public, which explains its popularity, but its positive class consists of prize draws and ringtone offers rather than the scams we target.

Deep learning on the same corpus. Roy, Singh and Banerjee trained CNN and LSTM models on the corpus and reported up to 99.44% accuracy [6]. Ahmadi *et al.* compared six classifiers with TF-IDF and bag-of-words features on a Kaggle copy and reported accuracies between 0.91 and 0.95, although some values in their text and tables disagree [13]. Labonne and Moran showed that a fine-tuned Flan-T5 model beats classical baselines for e-mail spam, especially with little training data [12].

Smishing detection. Sonowal and Kuppusamy proposed SmiDCA, which selects content features for machine-learning anti-smishing classifiers [8]. Mishra and Soni combined SMS content analysis with the behaviour of embedded URLs and reported 96.29% accuracy [7]. Goel *et al.* extracted smishing messages from the SMS Spam Collection, added 71 web messages, normalised slang and trained Naive Bayes, reaching 96.2% accuracy and a 3.87% false-positive rate [9].

Report-based corpora. Timko and Rahman released 1,062 validated Smishtank submissions with URL and sender metadata [10]. Agarwal *et al.* mined five public forums from 2017 to 2024, labelled scam type, brand and lure with GPT-4o, and

released 33,869 labelled messages in 66 languages [1]. They state that user reports carry no verification. The same group analysed 1.2 million UK operator reports from 2024 and found that 35.12% of unique reported texts were spam rather than scams [2]. MOZ-Smishing provides labelled genuine and fraudulent mobile-money messages for Mozambique [11].

Surveys. Pritom *et al.* reviewed 119 smishing papers and found datasets "often limited in scale, diversity, and temporal coverage", along with imbalanced data, synthetic evaluation data and subjective labelling of non-spam messages [3].

Methodology. Northcutt, Jiang and Chuang showed that disagreements between cross-validated predictions and given labels locate many label errors (confident learning) [14]. Lee *et al.* showed that near-duplicates inflate evaluation of language models, with train-test overlap in over 4% of standard validation sets [15]. Kapoor and Narayanan list duplicate data across train and test among the most common leakage errors in applied machine learning [16]. MinHash, introduced by Broder [17], is the standard tool for finding near-duplicate documents at scale.

Table 2.1 summarises the most relevant studies.

Table 2.1: Summary of related work

Work	**Technique**	**Dataset**	**Reported result**	**Limitation for our problem**
Almeida *et al.*, 2013 [5]	Many classifiers; linear SVM preferred	SMS Spam Collection, UK, 5,574	97.64% acc., 0.18% ham blocked	Promotional spam, not Indian scams
Roy *et al.*, 2020 [6]	CNN, LSTM	Same corpus	99.44% acc.	Same corpus, no cross-domain test
Mishra, Soni, 2020 [7]	Content + URL behaviour	SMS datasets	96.29% acc.	Depends on URLs
Goel *et al.*, 2024 [9]	Slang normalisation + Naive Bayes	Public smishing subset + 71 web messages	96.2% acc., FPR 3.87%	362 smishing messages
Timko, Rahman, 2024 [10]	Data release	Smishtank, 1,062	No classifier	No genuine class
Agarwal *et al.*, 2025 [1]	GPT-4o labelling	33,869 reports	No classifier	Reports unverified; no genuine class
Pritom *et al.*, 2026 [3]	Systematic review	119 papers	Not applicable	Identifies the dataset gap

2.3 Research Gap Identification

The survey points to four gaps.

1. **No verified genuine Indian class.** Classical corpora are not Indian; report-based corpora contain only reported messages. No public resource pairs Indian scams with Indian genuine messages under checked labels.
2. **Report labels are taken at face value.** Report-based corpora are used as if every report were a scam, although their own authors warn that reports are unverified and the UK study found large fractions of spam and OTPs among reports.
3. **Evaluation ignores campaigns.** Scam campaigns repeat one template many times. Random splits put copies of the same template on both sides, which inflates results, and none of the surveyed SMS studies controls for this.
4. **Explanation and rules are not evaluated independently.** Hybrid systems add rules, but their benefit is usually measured on messages chosen by the same developers.

2.4 Mapping of Objectives with Research Gap Identification

Table 2.2: Mapping of objectives to research gaps

Objective	**Gap addressed**
1. Build an Indian dataset with checked labels	Gaps 1 and 2: genuine Indian class; audit of report labels
2. Evaluate under a leakage-free protocol	Gap 3: campaign-aware split
3. Combine classifier with interpretable rules and measure their value	Gap 4: rules evaluated on independent real data
4. Deliver a deployed application	Practical use of the findings
5. Reproducibility	Supports verification of all four gaps

Chapter 3

Software and Hardware Requirement Specification

3.1 Functional Requirements

1. **Quick check.** A visitor can paste a message on the home page and receive a verdict, risk score, highlighted trap phrases and a short explanation without creating an account.
2. **Account management.** Users can register and sign in (with a captcha), or open a demo account, and can update or delete their profile and export their data.
3. **Full analysis.** A signed-in user can analyse text, a URL, an e-mail, a screenshot, a PDF or a QR code. The result includes the verdict, the classifier probability, the final risk score, the Scam X-Ray stages, the TrustLens report and recommended actions.
4. **Explanation.** For each result the system shows the matched phrases with their scam stage (hook, pressure, request, cash-out), evidence for and against, a similar known pattern with its source when one is accepted by the retrieval evaluator, and how a genuine message would differ.
5. **AI copilot.** Users can ask follow-up questions in English or Hindi; answers come from a hosted language model with per-visitor and daily limits, and fall back to built-in answers when the limit is reached.
6. **Safety tools.** QR code check, a "before you pay" check for UPI requests, a file check that detects disguised Android apps, a what-if view that shows how each cue changes the score, and a chat timeline that finds the turning point of a conversation.
7. **History, cases and reports.** Users can browse and search past scans, group scans into investigation cases with notes, give feedback on verdicts, download a PDF report and report a scam to the community view.
8. **Analytics.** Users see totals, verdict share and score trends; a public page shows aggregated scam trends.
9. **Administration.** Administrators can view registered model versions and their metrics and run a robustness test.

3.2 Non-functional Requirements

- **Performance.** Classification of one message takes about 2 ms, and about 4 ms including the intent layer, on a two-core server; page responses are compressed and cached.
- **Reliability.** The model is loaded once at start-up from a versioned registry; if the configured database is unreachable, the service starts on a local fallback database instead of failing. An automated job checks the live site every morning.
- **Security.** Passwords are hashed, access uses signed tokens with refresh, endpoints are rate-limited, request bodies are size-limited, and security headers are set.
- **Privacy.** The quick check stores nothing, and contributed messages are masked before they are stored.
- **Usability.** English and Hindi interface, an elder mode with larger text, keyboard-accessible controls, and layouts that work at phone width.
- **Maintainability.** Training and serving share the same preprocessing code; 818 automated back-end and ML tests run on every change.

3.3 Hardware Requirements

Table 3.1: Hardware requirements

Component	Requirement
Development machine	Any laptop with 8 GB RAM and a recent 64-bit CPU; no GPU is needed
Training	Under 3 seconds for the production model on a two-core CPU
Server	One small cloud instance (512 MB RAM is sufficient for the model, 1.6 MB on disk)
Client	Any browser on a phone or computer

3.4 Software Requirements

Table 3.2: Software requirements

Layer	**Software**
Language	Python 3.10+ (tested on 3.13), TypeScript 5.5
Machine learning	scikit-learn 1.7, NumPy, pandas, SciPy, joblib, datasketch (dataset build), matplotlib (figures)
Back end	FastAPI 0.115, Uvicorn, SQLAlchemy 2.0, Alembic, Pydantic 2, SlowAPI, python-jose, passlib, ReportLab, Tesseract OCR
Front end	Next.js 14.2, React 18.3, Tailwind CSS 3.4, Recharts, Zod, Vitest
Database	PostgreSQL in production, SQLite locally
Deployment	Render (back end and front end), GitHub Actions for CI and the daily live check
Language model	Google Gemini through its API for the copilot (optional)

3.5 Cost Estimation

The project was built and deployed with free resources. Development used the team's own laptops. The application runs on Render's free tier, the copilot uses a free API tier with a daily request cap, and all libraries are open source. The datasets are free to use under their CC BY 4.0 and MIT licences. The recurring cost of the deployed prototype is therefore zero. A production deployment for many users would need a paid always-on instance and a managed database, priced according to the provider's current plans.

Chapter 4

Methodology

4.1 Tools and Techniques

- **TF-IDF** weights a term by how often it occurs in a message and how rare it is across messages [18]. We use word unigrams and bigrams together with character 2–5-grams.
- **Logistic regression** gives a linear, interpretable model whose coefficients show which words push a message towards "scam".
- **MinHash with locality-sensitive hashing** [17] groups near-identical messages into campaigns.
- **Confident learning** [14] guides a second review of labels that a cross-validated model contradicts.
- **Regular-expression intent rules** detect requests for OTPs, payments to receive money, remote-access apps, legal threats and similar tactics, and protective signals such as "do not share your OTP".
- **Bootstrap confidence intervals and McNemar's test** quantify uncertainty and compare models on the same messages.
- **scikit-learn** [19] implements the models; FastAPI and Next.js implement the application.

4.2 Data Set With Explanation

4.2.1 Need for an Indian dataset

Indian scam messages and Indian genuine service messages share most of their vocabulary, and public smishing data contains only reported messages with unverified labels. A detector for Indian users therefore needs real Indian genuine messages, checked labels for reported messages and current scam types. ScamGuard-IN-2026 was built for this purpose, and it is the only dataset used for training and evaluation in this project.

4.2.2 Sources

Table 4.1: Sources of ScamGuard-IN-2026

Source	Licence	Genuine	Scam	Use
R1: IMC 2025 smishing collection, India subset, audited [1]	CC BY 4.0	963	3,987	train, development, test
R2: India Spam SMS collection, audited	MIT	1,694	6	train, development, test
R3: 2025–26 advisory and news texts	Quoted with source	1	11	test only
R4: Team-contributed 2026 SMS, masked	Team consent	–	–	test only (being collected)
S: Synthetic templates	Project	2,950	1,787	training only
Total		**5,608**	**5,791**	**11,399 messages**

R1 is the part of the IMC 2025 collection that concerns India: English or Hindi messages whose sender network is Indian or whose text names an Indian bank, payment app, identity document, operator or currency. The collection's "spam" category, which holds advertisements, was excluded. **R2** contains Indian personal, work and promotional messages. **R3** holds twelve messages from 2025–26 advisories and news, stored with source URL and date, including one genuine operator notice from a team member's phone. **R4** will hold messages received by the team in 2026, masked for personal data. **S** covers scam types that are new or rare in the real data, such as digital arrest calls, e-challan and wedding-invitation APKs, task-based jobs, stock-tip groups, Hindi electricity threats and fake "amount credited" messages, and genuine Indian formats such as bank alerts, OTPs with warnings, delivery and ride codes, operator and government notices and personal chat. Synthetic messages are never used for testing.

4.2.3 Attributes

Each row of the dataset has seven attributes: text (the message), label (scam or legitimate), source (R1–R4 or synthetic), subtype (scam type or genuine type), is_synthetic, campaign (near-duplicate group) and split (train, development, test or 2026 test).

4.2.4 Label audit

The IMC 2025 messages are messages that users *reported*. When we read a sample, about a third of the groups were genuine. We therefore checked every group:

1. 1,253 groups had a strong fraud cue (an account threat plus a KYC or call instruction, a prize plus a fee, a request to share an OTP) and no genuine cue, and were confirmed as scams by rule. A spot check of 40 found no error.
2. The other 1,669 groups were read one by one: 583 were scams, 867 genuine and 219 unclear, which were dropped.
3. A cross-validated model then flagged 44 rows whose label it strongly contradicted; each was re-read, giving 19 corrections and 7 removals.

After the audit, 963 of 4,950 retained IMC rows (19.5%) are genuine: debit and credit alerts, OTP messages with a warning, shipping updates, premium reminders and bank advertisements. Six disguised scams were also found among the R2 promotions. Every decision is stored in audit files in the repository.

4.2.5 Campaigns and splits

Messages are grouped into campaigns when their MinHash-estimated Jaccard similarity over word 3-shingles is at least 0.6. The 6,662 real messages form 4,085 campaigns; the largest has 128 messages and the largest 1% of campaigns hold 14% of real messages. Each campaign goes entirely to training (60%), development (15%) or test (25%), chosen by a hash of its first message. Table 4.2 gives the final counts.

Table 4.2: Dataset splits

Split	Genuine	Scam	Content
Train	4,581	4,064	real and synthetic
Development	361	507	real only
Test	665	1,209	real only, unseen campaigns
2026 test	1	11	advisories (team data pending)

4.3 Flow Chart Of Methodology From Start To End

Figure 4.1 shows the methodology from data collection to deployment, and Figure 4.2 the architecture of the deployed system.

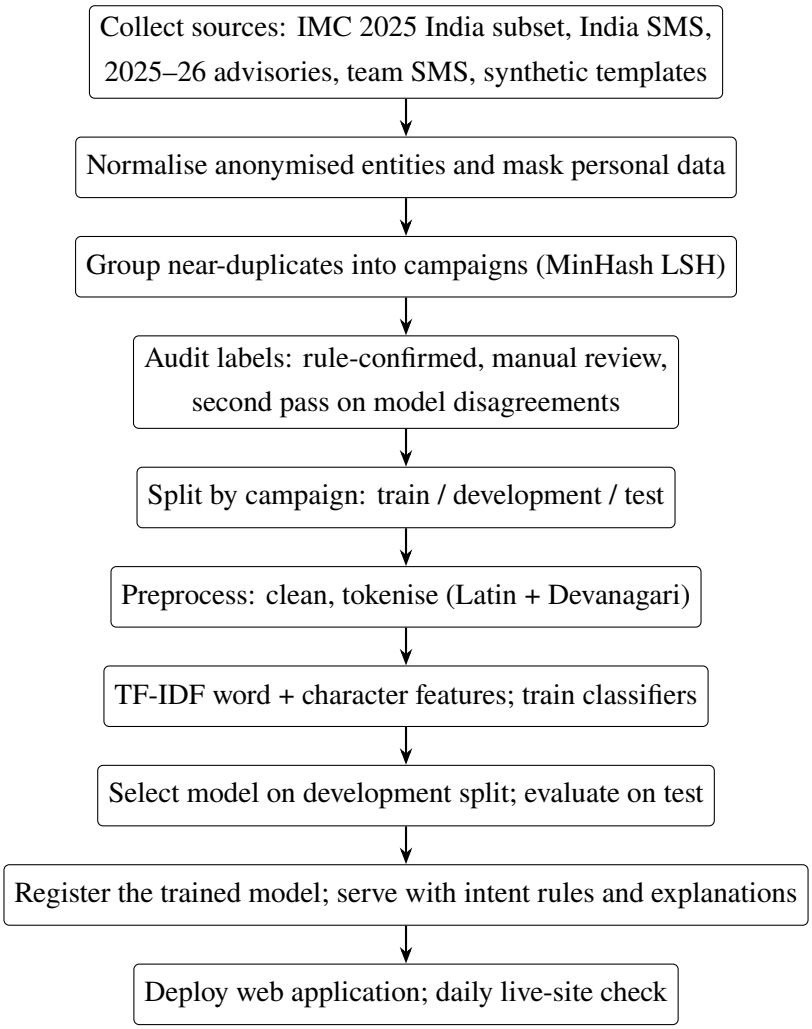


Figure 4.1: Flow of the methodology

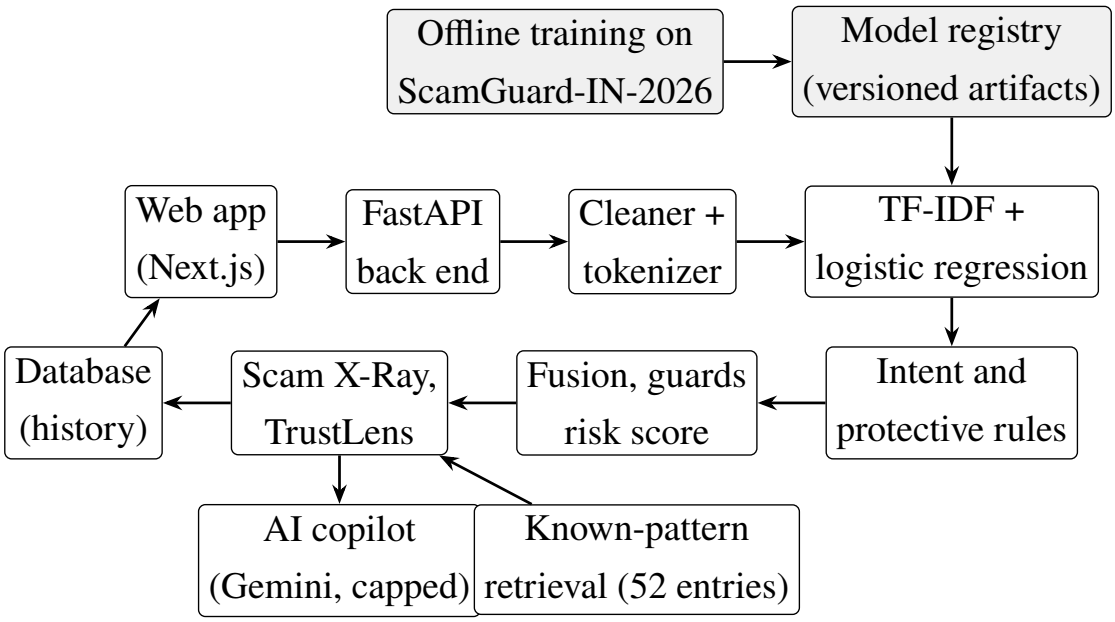


Figure 4.2: Architecture of the deployed system. Shaded parts run offline.

4.4 Explanation Of Methodology In Details With Equations And Formulae

4.4.1 Problem definition

For a message x, the detector computes a score $s(x) \in [0, 1]$ and predicts

$$\hat{y} = \begin{cases} 1 \text{ (scam)} & \text{if } s(x) \geq 0.5 \\ 0 \text{ (legitimate)} & \text{otherwise.} \end{cases} \tag{4.1}$$

4.4.2 Preprocessing

The cleaner replaces e-mail addresses, URLs (including bare links such as bit.ly/x) and phone numbers with placeholder tokens, lowercases the text, removes punctuation while keeping Devanagari vowel signs, and replaces remaining numbers with a number token. The tokenizer keeps Latin and Devanagari words, removes English stop words and applies a light suffix stemmer. The same code runs in training and in the live service, which rules out train-serve skew. Devanagari words are kept intact, so Hindi messages contribute features just as English ones do.

4.4.3 Feature extraction

For a term t in message d, with N training messages and document frequency $\mathrm{df}(t)$, the TF-IDF weight is

$$w_{t,d} = \left(1 + \log \mathrm{tf}_{t,d}\right) \cdot \left(\log \frac{1+N}{1+\mathrm{df}(t)} + 1\right), \tag{4.2}$$

and each message vector is normalised to unit length. Word features use unigrams and bigrams (up to 30,000, minimum document frequency 2, maximum 95%); character features use 2–5-grams inside word boundaries (up to 60,000, minimum document frequency 2). The two vectors are concatenated into $\phi(x)$ with 35,148 dimensions after pruning. Character n-grams let the model recognise *K.Y.C*, *bl0cked* or *S I M* as the words they imitate.

4.4.4 Classifier

Logistic regression estimates

$$p(x) = \sigma\left(\mathbf{w}^{\top}\phi(x) + b\right) = \frac{1}{1 + e^{-(\mathbf{w}^{\top}\phi(x)+b)}}, \tag{4.3}$$

by minimising the class-weighted, ℓ_2-regularised log-loss

$$\min_{\mathbf{w},b} \ \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \alpha_{y_i}\Big[- y_i \log p(x_i) - (1 - y_i) \log\left(1 - p(x_i)\right)\Big], \tag{4.4}$$

with $C = 16$ and class weights $\alpha_c = n/(2n_c)$. The per-word contribution to a verdict is $w_t\, w_{t,d}$, which the application shows as the classifier's top words.

4.4.5 Intent evidence and fusion

Thirty-one patterns detect 26 scam intents with hand-set weights w_i and ten protective signals with weights v_j. With $E^{+} = \sum_{i \in I(x)} w_i$ (halved when only weak context cues match) and $E^{-} = \sum_{j \in P(x)} v_j$ (halved when $E^{+} \geq 0.5$), the final score is

$$s(x) = \sigma\Big(\mathrm{clip}\left(\mathrm{logit}\, p(x), -4, 4\right) + 5\,(E^{+} - E^{-})\Big), \tag{4.5}$$

and it is capped at 0.15 when the message has the shape of a genuine transaction alert or service notice without any blocking cue (an install or payment request, a call to action on a credited amount, a private phone number). A "decoy" rule catches messages that say "never share your OTP" and then ask for it.

4.4.6 Evaluation metrics

With scam as the positive class:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \tag{4.6}$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \qquad \text{False-alarm rate} = \frac{FP}{FP + TN}. \tag{4.7}$$

A false positive is a genuine message wrongly flagged, which is the error users notice first. Confidence intervals come from 1,000 bootstrap resamples, and paired comparisons use McNemar's test on the messages two models classify differently.

4.4.7 Inference procedure

Algorithm 1 Scoring one message in the live service

Require: message x; frozen vectorizer and model; intent rules

$t \leftarrow \textsc{Tokenize}(\textsc{Clean}(x))$
$p \leftarrow \sigma(\mathbf{w}^\top \phi(t) + b)$
find matched intents I and protective signals P
if a sentence asks for an OTP after a "never share" warning **then**
    add decoy intent
end if
$s \leftarrow \sigma(\text{clip}(\text{logit}\, p, \pm 4) + 5(E^+ - E^-))$
if genuine alert or notice with no blocking cue **then**
    $s \leftarrow \min(s, 0.15)$
end if
return verdict $\mathbf{1}[s \geq 0.5]$, p, s, highlighted phrases, explanation

Chapter 5

Results and Discussion

5.1 Experimental setup

Models were selected on the development split and then trained on the training and development splits together (9,513 messages). All results below are on the real test split: 1,874 messages (665 genuine, 1,209 scams) from campaigns never seen in training. We also report a 259-message challenge set that the team wrote while designing the rules; because the same people wrote both, it is a development check, not an independent test. Experiments ran on a two-core Intel Xeon at 2.8 GHz with Python 3.13 and scikit-learn 1.7.2. One script produces every table.

5.2 Performance of the deployed system

Table 5.1 and Figure 5.1 give the main result: all metrics of the deployed system on the real test set, next to the classifier alone and the learned fusion weights described in Section 5.6.

Table 5.1: Real test set, 1,874 messages from unseen campaigns (%; FAR = false-alarm rate)

System	Acc.	Prec.	Rec.	F1	FAR	AUC
Classifier only	97.5	98.6	97.6	98.1	2.6	0.997
Deployed system (hand rules)	97.1	98.2	97.2	97.7	3.2	0.995
Learned fusion	**97.8**	**98.7**	**97.8**	**98.3**	**2.3**	0.997

The deployed system catches 1,175 of the 1,209 test scams and passes 644 of the 665 genuine messages. The 95% confidence interval of its accuracy is 96.3–97.8%, and of its false-alarm rate 2.0–4.6%. On the twelve 2025–26 advisory messages it detects 10 of the 11 scams and passes the one genuine operator notice (F1 95.2%); this set is too small for a firm conclusion.

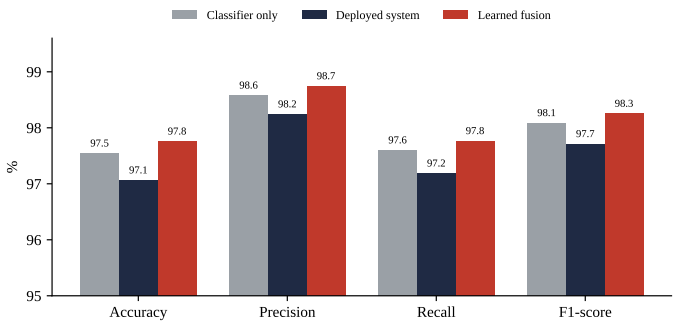


Figure 5.1: Accuracy, precision, recall and F1-score on the real test set (vertical axis starts at 95%)

5.3 Comparison of models and features

Table 5.2: Classifiers on the real test set (%); "Challenge" is F1 on the 259 team-written messages

Model	Features	F1	False alarms	Challenge F1
Naive Bayes	word	96.2	6.2	88.0
Naive Bayes	word + char	95.5	6.5	86.6
Logistic regression	word	98.0	2.7	89.9
Logistic regression	char	96.9	4.8	86.8
Logistic regression	**word + char**	**98.1**	**2.6**	90.2
Linear SVM	word	97.7	5.6	91.6
Linear SVM	word + char	98.2	3.9	92.0
Random forest	word	97.5	2.7	73.7
Random forest	word + char	97.6	3.3	73.9

Logistic regression with word and character features gives the lowest false-alarm rate among the strong models with an F1 of 98.1%. The SVM's F1 is 0.1 points higher, but it flags 3.9% of genuine messages. We chose logistic regression, which also gives readable word-level explanations.

5.4 Effect of the label audit

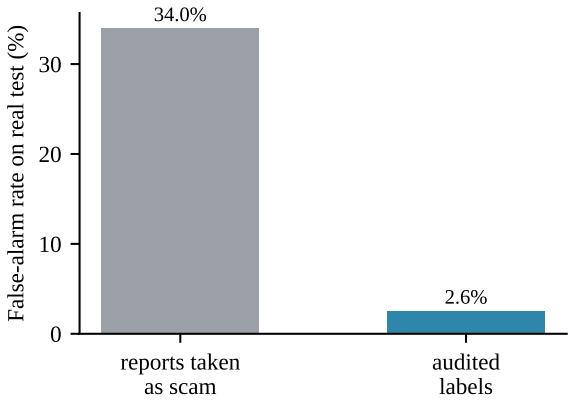


Figure 5.2: False-alarm rate with report labels and with audited labels

Training the same model with every reported message labelled as a scam gives 99.4% recall but flags 34.0% of genuine messages. With audited labels the false-alarm rate is 2.6% at 97.6% recall. The difference is highly significant (210 against 23 discordant messages, McNemar $p < 10^{-30}$). This is the clearest result of the project: checking the labels mattered far more than choosing the algorithm.

5.5 Effect of the split and of synthetic data

With a random split of the real data, 793 test messages had a near-duplicate in training. The random split gave an F1 of 98.3% and a 0.8% false-alarm rate; the campaign-aware split gave 97.7% and 1.4%. The random split hides nearly half of the false alarms.

Table 5.3: Effect of the training data (logistic regression, word + char, %)

Training data	Real F1	Real FAR	Challenge recall	Challenge F1
Synthetic only	87.4	31.1	86.4	90.3
Real only	97.7	1.4	35.6	51.2
Real + synthetic	98.1	2.6	82.2	90.2

Synthetic messages alone do not resemble real Indian SMS closely enough. Real messages alone miss most of the newest scam types. Together they keep real-data F1 at 98.1% and raise challenge F1 from 51.2% to 90.2%, at the cost of a slightly higher false-alarm rate.

5.6 Value of the intent rules

Table 5.4: Classifier alone and with fusion (%)

System	Real F1	Real false alarms	Challenge F1
Classifier only	98.1	2.6	90.2
Hand-weighted rules (deployed)	97.7	3.2	95.6
Learned fusion weights	98.3	2.3	91.2

On real messages from unseen campaigns, the hand-weighted rules do not improve accuracy and add a few false alarms. On the team's own challenge messages they look much better. Learned fusion weights are significantly better than the hand-set ones on real data ($p = 0.002$). We keep the rules in the application because they produce the phrase-level explanations and cover 2025–26 scam types that the real training data barely contains, but we do not claim that they improve accuracy.

5.7 Operating point and errors

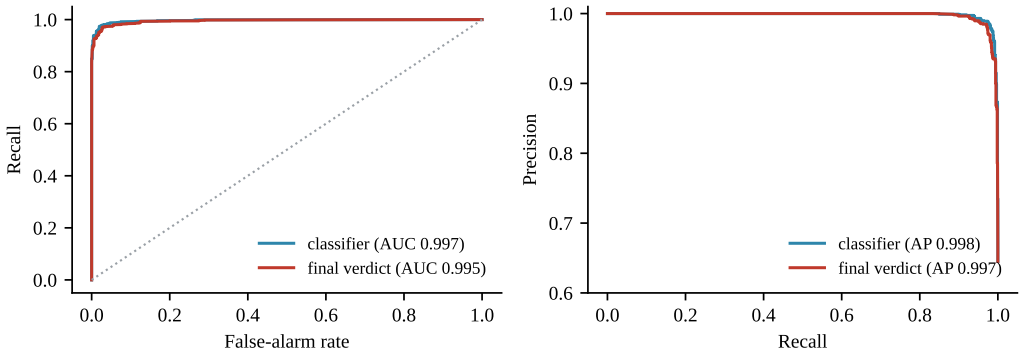


Figure 5.3: ROC and precision-recall curves on the real test set

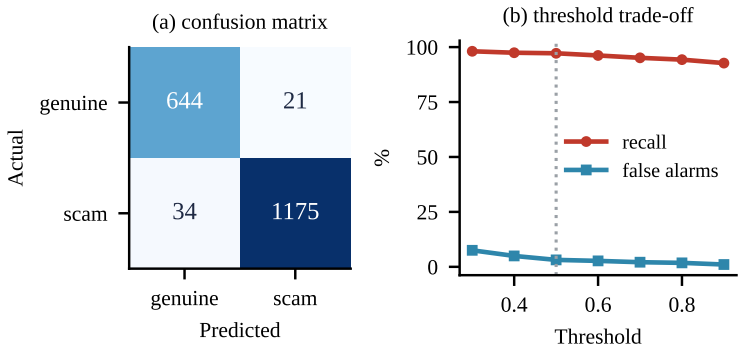


Figure 5.4: (a) Confusion matrix of the deployed system; (b) recall and false-alarm rate by threshold

The deployed score has a ROC-AUC of 0.995. At the default threshold it misses 34 of 1,209 scams and flags 21 of 665 genuine messages. Fifteen of these 21 are reported-but-genuine bank messages that ask the user to click a link to finish a KYC or card application, which are hard even for people. A threshold of 0.9 would lower false alarms to 0.8% at 92.5% recall. Recall is 97.7% on banking scams (895 of 916) and 92.3% on government-themed scams (24 of 26). Obfuscating keywords in 837 test scams lowered recall from 99.4% to 98.9% with word features and to 99.3% with word and character features.

The model and vectorizer take 1.6 MB on disk, training takes 2.7 seconds, and scoring one message takes about 2 ms (4 ms with the rules).

5.8 Application snapshots

Selected screens of the deployed application are shown in Appendix A.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

ScamGuard detects scam and fraud SMS for Indian users and explains each verdict. The main lesson of the project is about data. A model trained on user reports taken at face value flagged 34.0% of genuine messages. The same model trained on the audited ScamGuard-IN-2026 corpus flags 2.6% while catching 97.6% of scams, and the deployed system reaches 97.1% accuracy on real messages from unseen campaigns. Splitting the data by scam campaign gave a more honest estimate of false alarms than a random split, and comparing rules on independent real data showed that rules mainly serve explanation.

The result is a deployed application that answers in milliseconds, explains every verdict at phrase level, supports Hindi, and offers tools for the situations around a scam message: a QR code, a UPI request, a suspicious file, a conversation that turned into a request for money.

6.2 Future Work

- Complete the team-contributed 2026 test set (R4) and repeat the evaluation on it, since it is the only real test of 2025–26 scam types.
- Add a second annotator for the label audit and report agreement.
- Extend to Kannada, Tamil, Telugu and other Indian languages.
- Evaluate against adaptive attackers, including paraphrases generated by language models.
- Compare with small transformer models under the same campaign-aware protocol, within the memory budget of a free server.
- Use learned fusion weights in production once the 2026 test set confirms that they do not lose the coverage of the newest scam types.
- Offer the detector as an Android share target so that users can check a message without copying it.

REFERENCES

[1] S. Agarwal, A. Papasavva, G. Suarez-Tangil, and M. Vasek, "Fishing for smishing: Understanding SMS phishing infrastructure and strategies by mining public user reports," in *Proc. ACM Internet Measurement Conference (IMC '25)*, Madison, WI, USA, 2025, doi: 10.1145/3730567.3764431.

[2] S. Agarwal, G. Suarez-Tangil, and M. Vasek, "An overview of 7726 user reports: Uncovering SMS scams and scammer strategies," in *Proc. Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2026.

[3] M. M. A. Pritom, S. M. Sanjari, M. Mia, A. M. Shibli, S. M. M. Hossain, M. Ismail, and S. Xu, "Short message service (SMS) phishing attacks and defenses: A systematic review," arXiv:2604.11429, 2026.

[4] T. A. Almeida, J. M. Gómez Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: New collection and results," in *Proc. 11th ACM Symposium on Document Engineering*, Mountain View, CA, USA, 2011, pp. 259–262.

[5] T. A. Almeida, J. M. Gómez Hidalgo, and T. P. Silva, "Towards SMS spam filtering: Results under a new dataset," *International Journal of Information Security Science*, vol. 2, no. 1, pp. 1–18, 2013.

[6] P. K. Roy, J. P. Singh, and S. Banerjee, "Deep learning to filter SMS spam," *Future Generation Computer Systems*, vol. 102, pp. 524–533, 2020.

[7] S. Mishra and D. Soni, "Smishing Detector: A security model to detect smishing through SMS content analysis and URL behavior analysis," *Future Generation Computer Systems*, 2020.

[8] G. Sonowal and K. S. Kuppusamy, "SmiDCA: An anti-smishing model with machine learning approach," *The Computer Journal*, vol. 61, no. 8, pp. 1143–1157, 2018, doi: 10.1093/comjnl/bxy039.

[9] D. Goel, H. Ahmad, A. K. Jain, and N. K. Goel, "Machine learning driven smishing detection framework for mobile security," arXiv:2412.09641, 2024.

[10] D. Timko and M. L. Rahman, "Smishing Dataset I: Phishing SMS dataset from Smishtank.com," in *Proc. 14th ACM Conference on Data and Appli-*

cation Security and Privacy (CODASPY '24), Porto, Portugal, 2024, doi: 10.1145/3626232.3653282.

[11] F. D. M. A. Ali, S. M. Saide, R. Sousa-Silva, and H. Lopes Cardoso, "MOZ-Smishing: A benchmark dataset for detecting mobile money frauds," in *Proc. 6th Workshop on African Natural Language Processing*, Vienna, Austria, 2025, pp. 158–166, doi: 10.18653/v1/2025.africanlp-1.23.

[12] M. Labonne and S. Moran, "Spam-T5: Benchmarking large language models for few-shot email spam detection," arXiv:2304.01238, 2023.

[13] M. Ahmadi, M. Khajavi, A. Varmaghani, A. Ala, K. Danesh, and D. Javaheri, "Leveraging large language models for cybersecurity: Enhancing SMS spam detection with robust and context-aware text classification," arXiv:2502.11014, 2025.

[14] C. Northcutt, L. Jiang, and I. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *Journal of Artificial Intelligence Research*, vol. 70, pp. 1373–1411, 2021, doi: 10.1613/jair.1.12125.

[15] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini, "Deduplicating training data makes language models better," in *Proc. 60th Annual Meeting of the ACL*, Dublin, Ireland, 2022, pp. 8424–8445, doi: 10.18653/v1/2022.acl-long.577.

[16] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, 2023, doi: 10.1016/j.patter.2023.100804.

[17] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of SEQUENCES 1997*, Positano, Italy, 1997, doi: 10.1109/SEQUEN.1997.666900.

[18] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988, doi: 10.1016/0306-4573(88)90021-0.

[19] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Appendix A: Snapshots

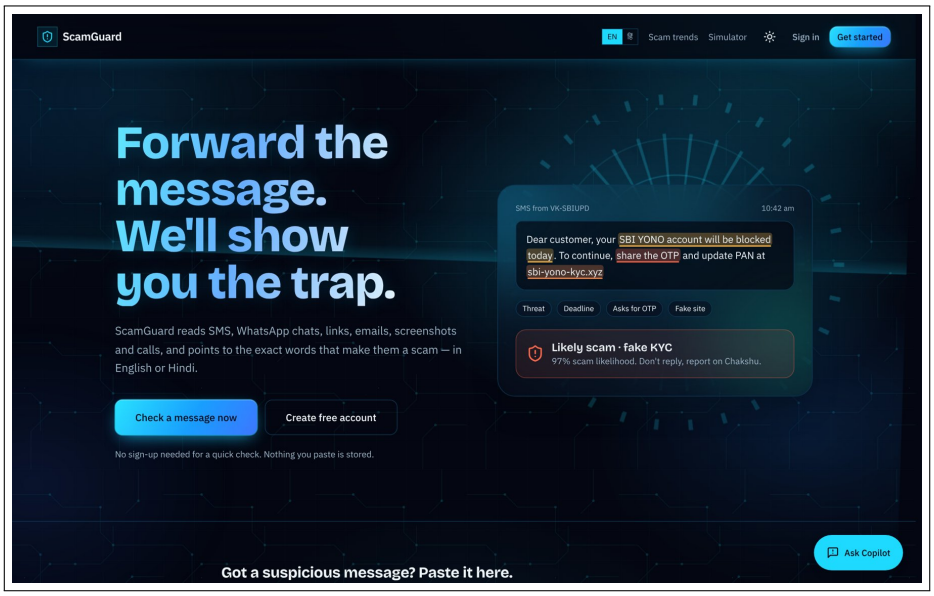


Figure A.1: Home page with the quick check

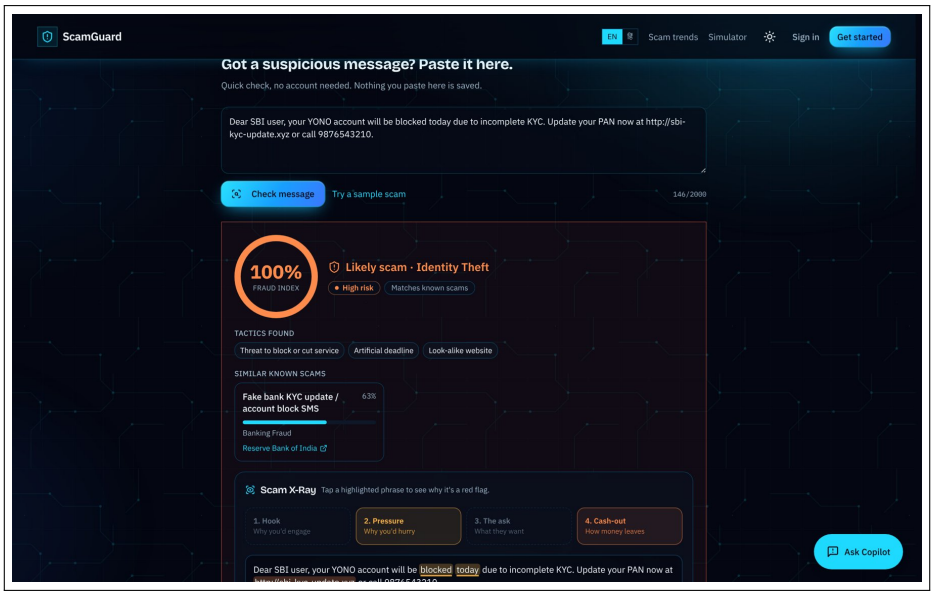


Figure A.2: Quick check of a fake KYC message: verdict, matched tactics, known pattern and Scam X-Ray

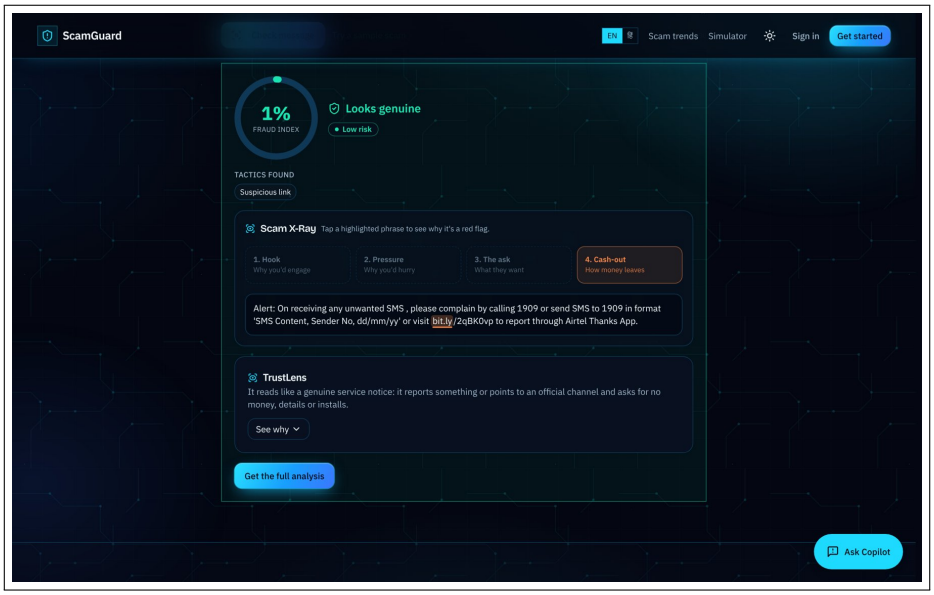


Figure A.3: Quick check of a genuine operator notice about the 1909 helpline: "Looks genuine" with the TrustLens reason

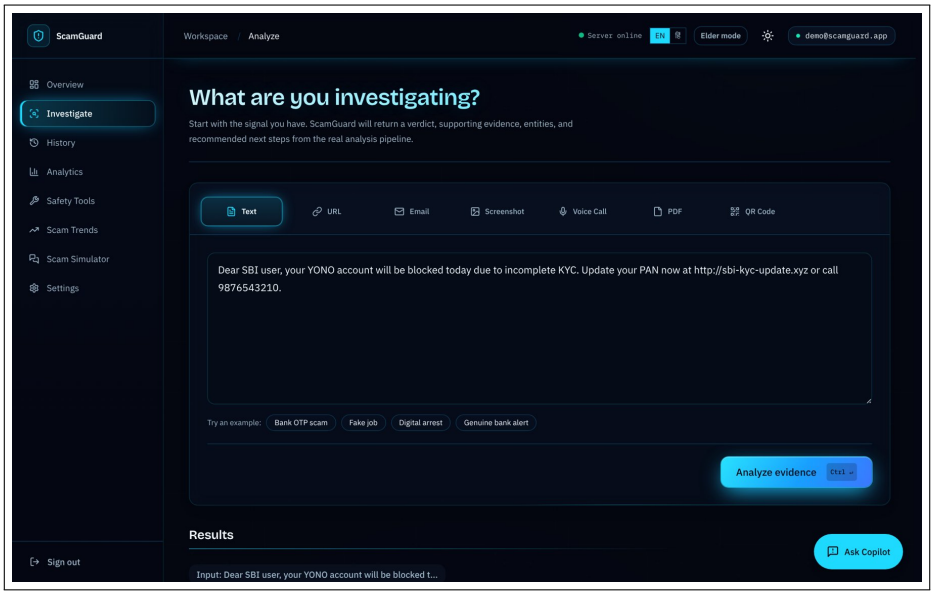


Figure A.4: Full analysis of a scam message for a signed-in user

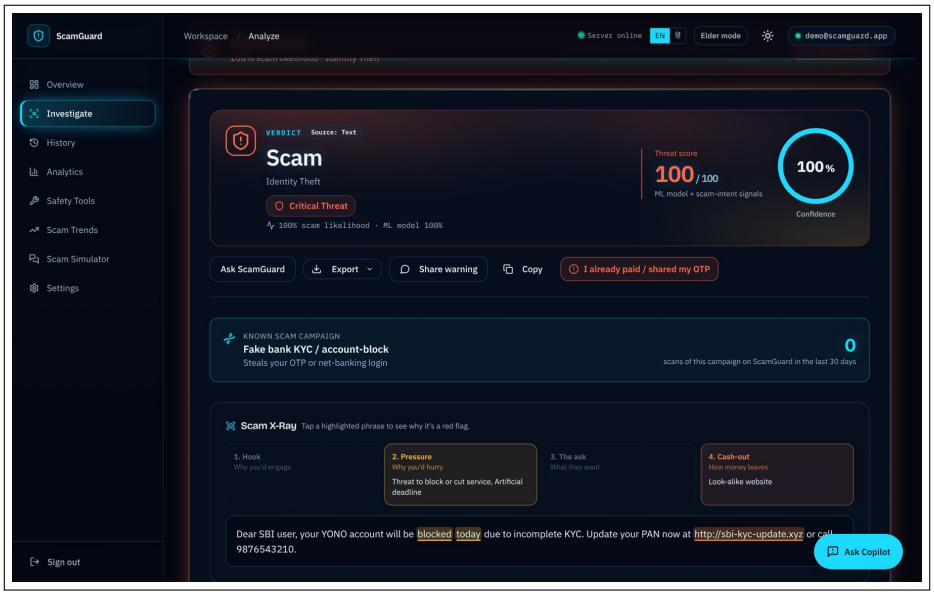


Figure A.5: Scam X-Ray: each risky phrase placed in the hook, pressure, request and cash-out stages

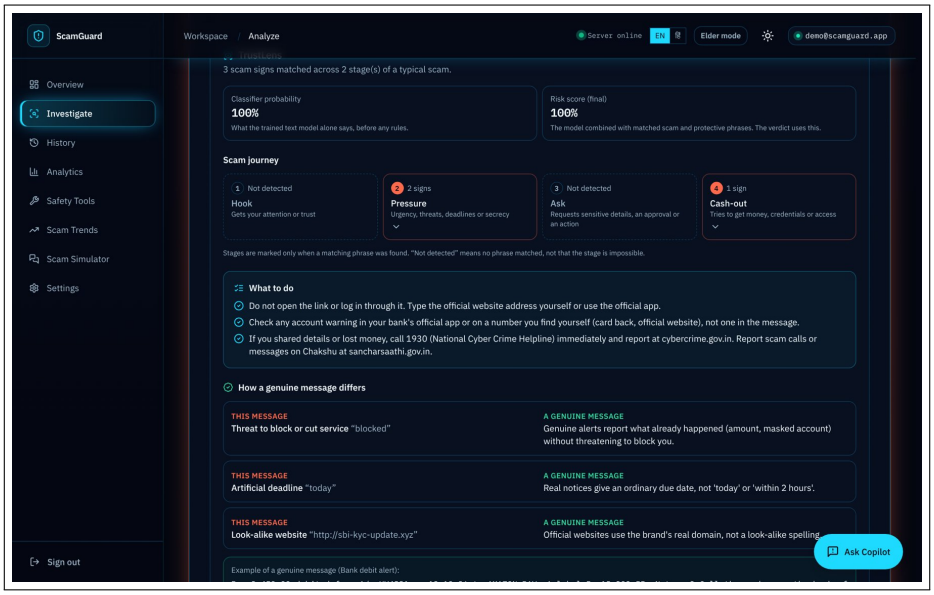


Figure A.6: TrustLens: scam journey, recommended actions and how a genuine message differs

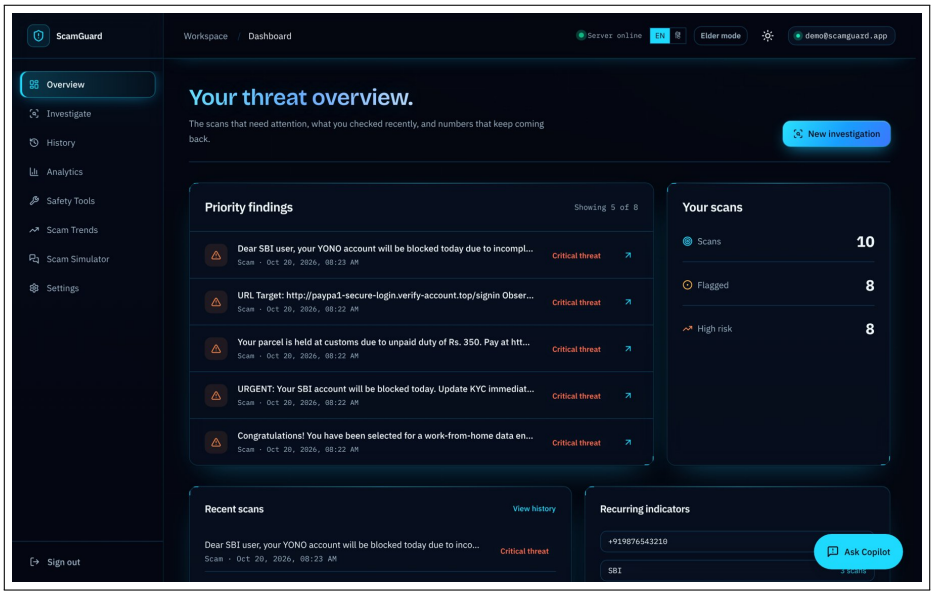


Figure A.7: Dashboard with priority findings and recent scans

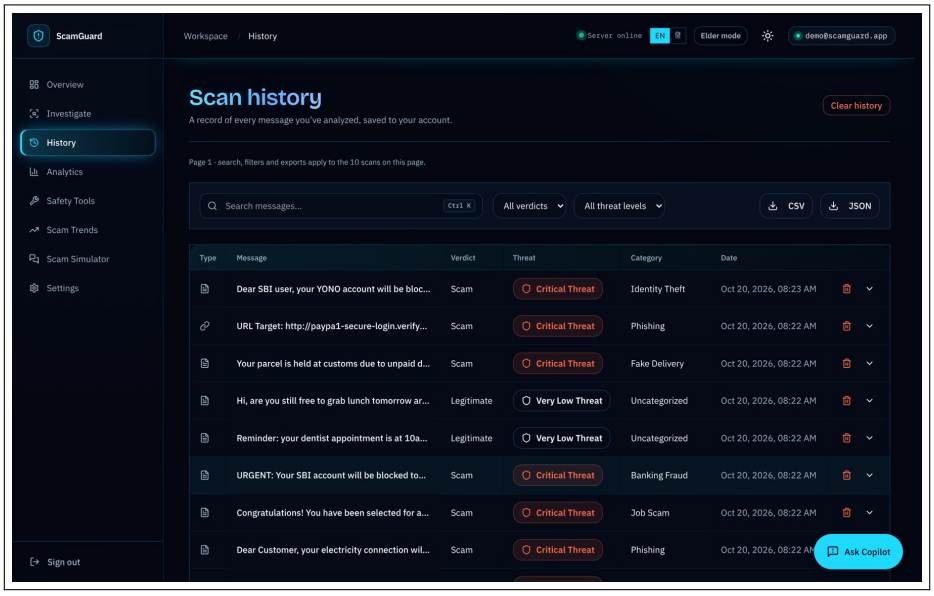


Figure A.8: Scan history with search, filters and export



Figure A.9: Analytics: verdict share, distribution and score trend

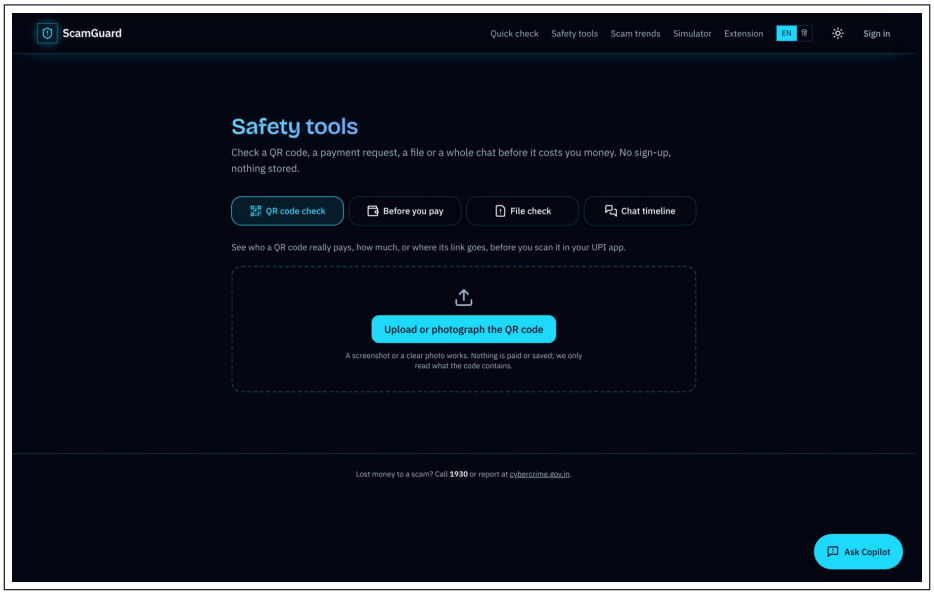


Figure A.10: Safety tools: QR code check, before-you-pay, file check and chat timeline

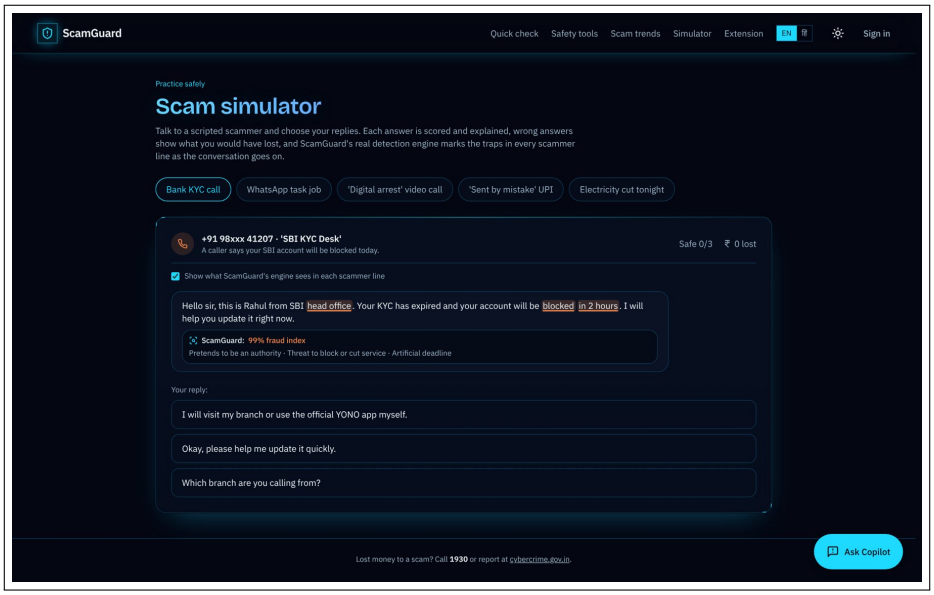


Figure A.11: Scam simulator for practice

Appendix B: Details of list of publications related to this project

Author Names: Kayarvizhy N, Priyam Thakur, Bhavya Goyal, Samir Chaudhary, Sana Subodh

Paper Title: ScamGuard: AI-Based Online Scam and Fraud Detection System

Name of the Conference or Journal: To be decided (manuscript prepared in IEEE conference format)

Place of the conference: –

Date of Conference: –

Appendix C: Details of Patent

No patent has been filed for this project.

Appendix D: Details of Funding

The project received no external funding. It was built with free cloud tiers and open-source software.

Appendix E: POs and PSOs Mapped

Table E.1: Programme Outcomes Mapping with Project

PROGRAMME OUTCOMES	**Level (1/2/3)**	**Justification if addressed**
PO1	3	Applies machine learning, NLP and statistics to classify scam messages and to evaluate the classifier.
PO2	3	Analyses why existing filters fail on Indian SMS and traces the failure to dataset labels and evaluation leakage.
PO3	3	Designs a dataset, a detector and a web application that meet accuracy, latency and explanation requirements.
PO4	3	Conducts controlled experiments with bootstrap intervals and significance tests on a campaign-held-out test set.
PO5	3	Uses scikit-learn, FastAPI, Next.js, MinHash, GitHub Actions and cloud deployment.
PO6	3	Protects users from financial fraud and points them to official reporting channels.
PO7	1	Runs on free, low-power infrastructure; the detector needs no GPU.
PO8	3	Masks personal data, respects dataset licences and reports limitations honestly.
PO9	3	Built by a team of four with shared responsibility for data, model, back end and front end.
PO10	3	Communicates results through a report, a research paper, a datasheet and an explanatory user interface.
PO11	2	Plans the work in phases and keeps the deployment within free-tier budgets.
PO12	2	Designs the dataset build so that new scam types and the 2026 test set can be added and re-evaluated.

Table E.2: Programme Specific Outcomes Mapping with Project

PROGRAMME SPECIFIC OUTCOMES	**Level (1/2/3)**	**Justification if addressed**
PSO1	2	Uses web APIs, cloud hosting, rate limiting and secure token-based communication.
PSO2	3	Applies software engineering practice: versioned models, automated tests, CI and reproducible builds.
PSO3	3	Integrates machine learning, NLP and information systems into a deployed, usable product.

Appendix F: Similarity report, AI Generated report

To be attached after the department's similarity and AI-content check.